RETAILSPHERE (PTY) LTD

Data Platform Programme

Physical Database Design

Step 9 Deliverable — DDL Scripts and Physical Design Specifications

Document ID	PDD-001
Version	**1.0 - Approved**
Date	23 August 2026
Author	Thozamile Mbalo – Data Solutions Architect
Reviewed by	Nomvula Mahlangu – Chief Data Officer
Target Platform	**Microsoft Fabric Warehouse**
Status	**Approved**

1. Purpose

This document defines the physical database design for the RetailSphere Gold layer data platform hosted on Microsoft Fabric Warehouse. It translates the conceptual dimensional model defined in DM-001 into deployable DDL scripts including CREATE TABLE statements, primary and foreign key constraints, and distribution strategies. PDD-001 serves as the primary technical reference for the engineering team during the Gold layer build phase (Step 15).

2. Scope

This document covers the physical design of all 9 dimension tables and 7 fact tables defined in DM-001 across 6 business domains, deployed within the gold schema in Microsoft Fabric Warehouse. It includes DDL scripts, constraint definitions, and distribution strategy for each table.

This document does not cover Bronze or Silver layer physical design – those are addressed in DID-001 (Step 10 – Data Integration Design). Pipeline orchestration and deployment automation are addressed in Step 16.

3. Target Platform

The RetailSphere Gold Layer is hosted on Microsoft Fabric Warehouse, Microsoft's unified data lake storage layer accessible by all Fabric experiences.

Microsoft Fabric Warehouse was selected for the Gold layer for the following reasons:

1. Azure-native alignment: RetailSphere is designed as an Azure-native platform. Fabric is Microsoft's strategic direction for modern data platforms on Azure, with active investment and roadmap commitment.
2. Unified storage via OneLake: All fabric experiences – Warehouse, Lakehouse, Notebooks, Power BI – read from the same OneLake storage. No data movement or distribution is required between the Gold layer and the Power BI reporting layer.
3. T-SQL compatibility: Fabric Warehouse uses T-SQL syntax, directly compatible with the team's existing SQL Server expertise. Temp tables and CTEs are supported.
4. Automatic data optimization: Fabric Warehouse applies Delta Lake V-Order optimization automatically at write time. No manual index creation is required, reducing operational overhead.
5. Power BI native integration: Power BI connects to Fabric Warehouse natively without a gateway, simplifying the reporting layer deployment in Step 17.

FABRIC WAREHOUSE CONSTRAINTS:

The following SQL Server features are not supported in Fabric Warehouse and have been considered in the DDL design:

- Recursive CTEs – not supported.
- Triggers – not supported.
- Stored procedures with complex DML – limited support.
- Foreign key enforcement – constraints are informational only, not enforced at engine level.

4. Naming Conventions

All Gold layer objects in Microsoft Fabric Warehouse follow a consistent naming convention to ensure readability, maintainability, and alignment with Kimball dimensional modelling standards.

Schema:
All Gold layer tables are created within the gold schema.
Example: gold.DimProduct, gold.FactSales

Table Names:
Tables use a prefix followed by PascalCase naming.

Dimension tables: Dim prefix
Example: DimDate, DimProduct, DimCustomer

Fact tables: Fact prefix
Example: FactSales, FactInventorySnapshot, FactReplenishment

Column Names
All column names use lowercase
snake_case.
Example: product_key, sales_date_key, unit_price

Primary Key Constraints:
pk_[tablename] in lowercase.
Example: pk_dimproduct, pk_factsales

Foreign Key Constraints:
fk_[facttable]_[dimension] in lowercase.
Example: fk_factsales_product, fk_factsales_customer

Note: Foreign key constraints in Microsoft Fabric Warehouse are informational only. They document relationships for lineage and documentation purposes but are not enforced at the engine level. Data integrity is enforced at the engine level. Data integrity is enforced through dbt tests in the Silver layer transformation pipeline.

5. Gold Layer DDL Scripts

This section contains the CREATE TABLE DDL scripts for all 16 Gold layer tables deployed in the gold schema in Microsoft Fabric Warehouse. Tables are sequenced – dimensions first, then fact tables. Each script includes primary key and foreign key constraint definitions.

5.1 Dimension Tables

The following DDL scripts define all 9 dimension tables in the RetailSphere Gold layer.

5.1.1 DimDate

```
-- ============================================

-- DimDate

-- Pre-populated date dimension

-- No SCD Type 2 required
```

```
-- ============================================


CREATE TABLE gold.DimDate (
    date_key            INT         NOT NULL,
    date                DATE        NOT NULL,
    ansi_date           CHAR(10)    NOT NULL,
    sql_date            DATETIME    NOT NULL,
    day                 INT         NOT NULL,
    day_of_the_week     INT         NOT NULL,
    day_name            VARCHAR(9)  NOT NULL,
    day_of_the_year     INT         NOT NULL,
    week_number         INT         NOT NULL,
    month               INT         NOT NULL,
    month_name          VARCHAR(9)  NOT NULL,
    short_month_name    CHAR(3)     NOT NULL,
    quarter             CHAR(2)     NOT NULL,
    year                INT         NOT NULL,
    fiscal_week         INT         NULL,
    fiscal_period       CHAR(4)     NULL,
    fiscal_quarter      CHAR(3)     NULL,
    fiscal_year         CHAR(6)     NULL,
    week_day            TINYINT     NOT NULL,
    rsa_holiday         TINYINT     NOT NULL,
    short_trading_day   TINYINT     NOT NULL,
    month_end           TINYINT     NOT NULL,
    period_end          TINYINT     NULL,

    CONSTRAINT pk_dimdate
      PRIMARY KEY (date_key)
);
```

5.1.2 DimProduct

DimProduct is a conformed dimension implementing SCD Type 2 for price and status changes. Cross-reference columns for SAP, Shopify, and POS are nullable as a product may not exists in all three source systems at initial load. ADR-001 governs the identity resolution strategy for this dimension

```
-- ============================================
-- DimProduct
-- Conformed dimension — SCD Type 2
-- Sources: SAP ECC, Shopify, Oracle MICROS POS
-- ADR-001: Master Product Identifier Strategy
-- ============================================

CREATE TABLE gold.DimProduct (
    product_key          INT          NOT NULL,
    product_code         VARCHAR(20)  NOT NULL,
    name                 VARCHAR(30)  NOT NULL,
    description          VARCHAR(100) NULL,
    product_type         VARCHAR(20)  NULL,
    product_category     VARCHAR(30)  NULL,
    product_subcategory  VARCHAR(30)  NULL,
    unit_price           MONEY        NOT NULL,
    unit_cost            MONEY        NOT NULL,
    status               VARCHAR(15)  NOT NULL,
    sap_stock_code       VARCHAR(20)  NULL,
    shopify_sku          VARCHAR(50)  NULL,
    pos_barcode          VARCHAR(20)  NULL,
    supplier_code        VARCHAR(20)  NULL,
    effective_start_date DATE         NOT NULL,
    effective_end_date   DATE         NULL,
    is_current           BIT          NOT NULL,
    source_system_code   INT          NOT NULL,
    create_timestamp     DATETIME     NOT NULL,
    update_timestamp     DATETIME     NOT NULL,
```

```
    CONSTRAINT pk_dimproduct
        PRIMARY KEY (product_key)
);
```

5.1.3 DimCustomer

DimCustomer is the most complex dimension in the platform, implementing SCD Type 2 and sourcing identity data from six systems per ADR-002. The majority of the columns are nullable to accommodate the 66% of in-store transactions that are anonymous. POPIA compliance columns opted_out_status and is_anonymous are NOT NULL with safe default values – ensuring compliance is enforced at the physical layer regardless of data availability.

```
-- ============================================
-- DimCustomer
-- Conformed dimension — SCD Type 2
-- Sources: Salesforce CRM, Shopify,
--          Oracle MICROS POS, Marketing Cloud,
--          Meta, GA4
-- ADR-002: Unified Customer Identity Resolution
-- ADR-007: Contact Governance and Opt-out
-- ============================================

CREATE TABLE gold.DimCustomer (
    customer_key            INT          NOT NULL,
    customer_id             VARCHAR(50)  NOT NULL,
    customer_loyalty_number INT          NULL,
    first_name              VARCHAR(100) NULL,
    last_name               VARCHAR(100) NULL,
    gender                  CHAR(1)      NULL,
    email_address           VARCHAR(200) NULL,
    date_of_birth           DATE         NULL,
    id_number               VARCHAR(20)  NULL,
    address1                VARCHAR(50)  NULL,
    address2                VARCHAR(50)  NULL,
    address3                VARCHAR(50)  NULL,
    address4                VARCHAR(50)  NULL,
    city                    VARCHAR(200) NULL,
    province                VARCHAR(150) NULL,
    postal_code             VARCHAR(50)  NULL,
    country                 VARCHAR(250) NULL,
    phone_number            VARCHAR(30)  NULL,
    occupation              VARCHAR(100) NULL,
    household_income        VARCHAR(20)  NULL,
    date_registered         DATETIME     NULL,
    status                  VARCHAR(10)  NULL,
    salesforce_loyalty_id   VARCHAR(50)  NULL,
    shopify_customer_id     VARCHAR(50)  NULL,
    pos_receipt             VARCHAR(50)  NULL,
    mk_cloud_id             VARCHAR(50)  NULL,
```

```
    meta_pixel_id           VARCHAR(50)   NULL,
    ga4_client_id           VARCHAR(50)   NULL,
    opted_out_status        BIT           NOT NULL,
    popia_consent_date      DATE          NULL,
    is_anonymous            BIT           NOT NULL,
    customer_segment        VARCHAR(50)   NULL,
    effective_start_date    DATE          NOT NULL,
    effective_end_date      DATE          NULL,
    is_current              BIT           NOT NULL,
    source_system_code      INT           NOT NULL,
    create_timestamp        DATETIME      NOT NULL,
    update_timestamp        DATETIME      NOT NULL,

    CONSTRAINT pk_dimcustomer
        PRIMARY KEY (customer_key)
);
```

5.1.4 DimStore:

DimStore describes RetailSphere's 87 stores nationally. SCD Type 2 is applied to track store manager changes for historical sales performance attribution. Address and configuration field names are nullable as POS field names are assumed pending source system configuration in Step 13.

```
-- ============================================
-- DimStore
-- Domain-specific dimension — SCD Type 2
-- Source: Oracle MICROS POS
-- SCD Type 2 trigger: Store manager change
-- ============================================

CREATE TABLE gold.DimStore (
    store_key           INT           NOT NULL,
    store_id            VARCHAR(20)   NOT NULL,
    store_number        INT           NOT NULL,
    store_name          VARCHAR(100)  NOT NULL,
    store_type          VARCHAR(50)   NULL,
    region              VARCHAR(50)   NULL,
    store_address1      VARCHAR(50)   NULL,
    store_address2      VARCHAR(50)   NULL,
    store_address3      VARCHAR(50)   NULL,
    store_address4      VARCHAR(50)   NULL,
    city                VARCHAR(100)  NULL,
    province            VARCHAR(150)  NULL,
    postal_code         VARCHAR(20)   NULL,
    phone_number        VARCHAR(30)   NULL,
    store_manager       VARCHAR(100)  NULL,
    store_size_sqm      INT           NULL,
    trading_hours       VARCHAR(100)  NULL,
    is_active           BIT           NOT NULL,
    effective_start_date  DATE        NOT NULL,
```

```
    effective_end_date    DATE        NULL,
    is_current            BIT         NOT NULL,
    source_system_code    INT         NOT NULL,
    create_timestamp      DATETIME    NOT NULL,
    update_timestamp      DATETIME    NOT NULL,

    CONSTRAINT pk_dimstore
      PRIMARY KEY (store_key)
);
```

5.1.5 DimEmployee

DimEmployee describes RetailSphere employees who process sales transactions, primarily cashiers and store managers. SCD Type 2 is applied with store transfer and promotion as change triggers, enabling historical sales performance attribution to the correct employee role and location at time of sale. Salary data is deliberately excluded from the Gold layer and remains in the OLTP HR source system.

```
-- ============================================
-- DimEmployee
-- Domain-specific dimension — SCD Type 2
-- Source: Oracle MICROS POS
-- SCD Type 2 triggers: store transfer, promotion
-- Salary excluded — remains in OLTP HR system
-- ============================================

CREATE TABLE gold.DimEmployee (
  employee_key        INT         NOT NULL,
  employee_id         VARCHAR(20)   NOT NULL,
  first_name          VARCHAR(100)  NOT NULL,
  last_name           VARCHAR(100)  NOT NULL,
  gender              CHAR(1)       NULL,
  email_address       VARCHAR(200)  NULL,
  phone_number        VARCHAR(30)   NULL,
  date_of_birth       DATE          NULL,
  id_number           VARCHAR(20)   NULL,
  address1            VARCHAR(50)   NULL,
  address2            VARCHAR(50)   NULL,
  address3            VARCHAR(50)   NULL,
  address4            VARCHAR(50)   NULL,
  city                VARCHAR(200)  NULL,
```

```
    province            VARCHAR(150)  NULL,
    postal_code         VARCHAR(50)   NULL,
    country             VARCHAR(250)  NULL,
    employment_type     VARCHAR(50)   NULL,
    title               VARCHAR(100)  NULL,
    manager_id          VARCHAR(50)   NULL,
    store_id            VARCHAR(20)   NULL,
    department_name     VARCHAR(150)  NULL,
    effective_start_date  DATE        NOT NULL,
    effective_end_date    DATE        NULL,
    is_current          BIT           NOT NULL,
    source_system_code    INT         NOT NULL,
    create_timestamp    DATETIME      NOT NULL,
    update_timestamp    DATETIME      NOT NULL,

    CONSTRAINT pk_dimemployee
        PRIMARY KEY (employee_key)
);
```

5.1.6 DimSupplier

DimSupplier describes RetailSphere's 340 active suppliers and is used across the Inventory, Procurement, and Finance domains. SCD Type 1 is applied – supplier profile changes are overwritten as historical supplier contact details carry no analytical value. Contracted rates, lead times, and payment terms are held in FactSupplierContract rather than this dimension, as the supplier-to-product relationship is many-to-many.

```
-- ============================================
-- DimSupplier
-- Domain-specific dimension — SCD Type 1
-- Source: SAP ECC (LFA1 Vendor Master)
-- Used by: Inventory, Procurement, Finance
-- Contract terms held in FactSupplierContract
-- ============================================

CREATE TABLE gold.DimSupplier (
    supplier_key        INT         NOT NULL,
    supplier_code       VARCHAR(20)   NOT NULL,
```

```
    supplier_name       VARCHAR(100)  NOT NULL,
    email_address       VARCHAR(200)  NULL,
    phone_number        VARCHAR(30)   NULL,
    address1            VARCHAR(50)   NULL,
    address2            VARCHAR(50)   NULL,
    address3            VARCHAR(50)   NULL,
    address4            VARCHAR(50)   NULL,
    city                VARCHAR(100)  NULL,
    province            VARCHAR(150)  NULL,
    postal_code         VARCHAR(20)   NULL,
    region              VARCHAR(50)   NULL,
    country             VARCHAR(250)  NULL,
    contact_person      VARCHAR(100)  NULL,
    status              VARCHAR(20)   NOT NULL,
    supplier_category   VARCHAR(50)   NULL,
    source_system_code  INT           NOT NULL,
    create_timestamp    DATETIME      NOT NULL,
    update_timestamp    DATETIME      NOT NULL,

    CONSTRAINT pk_dimsupplier
      PRIMARY KEY (supplier_key)
);
```

5.1.7 DimCampaign

DimCampaign describes marketing campaigns across RetailSphere's three marketing platforms. SCD Type 1 is applied as campaign history is not required – a campaign's attributes are fixed once launched, and date change trigger a new campaign rather than a revision. The status column is system-calculated daily by the Silver layer based on comparison between end_date and current date, rather than being manually maintained in source systems.

```
-- ============================================
-- DimCampaign
-- Domain-specific dimension — SCD Type 1
-- Sources: Salesforce Marketing Cloud,
--          Meta Business Manager, GA4
-- status is system-calculated daily
-- ============================================
```

```
CREATE TABLE gold.DimCampaign (
    campaign_key        INT         NOT NULL,
    campaign_id         VARCHAR(20)   NOT NULL,
    campaign_name       VARCHAR(100)  NOT NULL,
    campaign_type       VARCHAR(50)   NULL,
    budget              MONEY       NULL,
    start_date          DATE        NOT NULL,
    end_date            DATE        NULL,
    status              VARCHAR(20)   NOT NULL,
    source_system_code  INT         NOT NULL,
    create_timestamp    DATETIME    NOT NULL,
    update_timestamp    DATETIME    NOT NULL,

    CONSTRAINT pk_dimcampaign
        PRIMARY KEY (campaign_key)
);
```

5.1.8 DimChannel

DimChannel is a small reference dimension of approximately seven rows describing RetailSphere's sales and marketing channels. It is not extracted from any source system – it is manually maintained and loaded once at platform setup, updated only when new channels are introduced. All columns are NOT NULL as the table is manually curated. The channel_type column must align with the attribution channel definitions established in ADR-006.

```
-- ============================================
-- DimChannel
-- Reference dimension — manually maintained
-- No source system extraction
-- Approximately 7 rows
-- ============================================

CREATE TABLE gold.DimChannel (
    channel_key         INT         NOT NULL,
    channel_code        VARCHAR(20)   NOT NULL,
    channel_name        VARCHAR(50)   NOT NULL,
    channel_type        VARCHAR(50)   NOT NULL,
    create_timestamp    DATETIME    NOT NULL,
    update_timestamp    DATETIME    NOT NULL,

    CONSTRAINT pk_dimchannel
        PRIMARY KEY (channel_key)
);
```

5.1.9 DimAccount

DimAccount describes RetailSphere's Chart of Accounts, providing the account type and category hierarchy required for financial statement grouping and subtotaling. SCD Type 2 is applied to preserve historical account classifications – this is an audit requirement, ensuring financial statements can be reproduced exactly as originally reported and signed off even if account classifications are subsequently changed. The account_type and normal_balance columns are NOT NULL as financial statement aggregation logic depends on both.

```
-- ============================================
-- DimAccount
-- Domain-specific dimension — SCD Type 2
-- Source: SAP ECC Chart of Accounts
-- SCD Type 2 for audit reproducibility
-- ============================================

CREATE TABLE gold.DimAccount (
    account_key         INT          NOT NULL,
    account_code        VARCHAR(20)  NOT NULL,
    account_name        VARCHAR(100) NOT NULL,
    account_type        VARCHAR(20)  NOT NULL,
    account_category    VARCHAR(50)  NULL,
    normal_balance      VARCHAR(10)  NOT NULL,
    is_active           BIT          NOT NULL,
    effective_start_date DATE        NOT NULL,
    effective_end_date  DATE         NULL,
    is_current          BIT          NOT NULL,
    source_system_code  INT          NOT NULL,
    create_timestamp    DATETIME     NOT NULL,
    update_timestamp    DATETIME     NOT NULL,

    CONSTRAINT pk_dimaccount
        PRIMARY KEY (account_key)
);
```

5.2 Fact Tables

This section defines the seven fact tables in the RetailSphere Gold Layer. Each fact table declares a surrogate primary key, foreign key constraints to its supporting dimensions, and where applicable a unique constraint on its degenerate dimension columns.

Foreign key and unique constraints in Microsoft Fabric Warehouse are informational only – they are stored as metadata for lineage, documentation, and uniqueness are enforced by dbt tests in the Silver layer transformation pipeline before data is committed to Gold. The constraints defined here document the intended data contract; dbt enforces it.

Foreign key referencing DimCustomer and DimStore permit a value of 0 rather than NULL, where no business entity applies – 0 references a designated ‘unknown’ member row in each dimension. This preserves the integrity of inner joins in the reporting layer, avoiding the silent row loss that NULL foreign keys would cause.

5.2.1 FactSales

FactSales is a transaction-grain fact table capturing one row per product line item per transaction. It is the highest-volume table in the platform and serves the Sales domain reporting requirements FR-SAL-001 through FR-SAL-005.

A surrogate primary key sales_key is used rather than a composite of dimension foreign keys, as the dimension combination is not unique – a customer may purchase the same product at the same store on the same day in separate transactions. The degenerate dimension columns order_id and line_number carry the business transaction identity and are declared as a unique constraint to prevent duplicate line loads.

customer_key is set to 0 for the 66% of in-store transactions that are anonymous. store_key is set to 0 for online Shopify orders where no physical store applies.

```
-- ============================================
-- FactSales
-- Transaction grain — one row per line item
-- Sources: Oracle MICROS POS, Shopify,
--          Salesforce CRM
-- ============================================

CREATE TABLE gold.FactSales (
    sales_key           BIGINT       NOT NULL,
    sales_date_key      INT          NOT NULL,
    customer_key        INT          NOT NULL,
    employee_key        INT          NOT NULL,
    channel_key         INT          NOT NULL,
    product_key         INT          NOT NULL,
    store_key           INT          NOT NULL,
    order_id            VARCHAR(50)  NOT NULL,
    line_number         INT          NOT NULL,
    quantity            INT          NOT NULL,
    unit_price          MONEY        NOT NULL,
    unit_cost           MONEY        NULL,
    sales_value         MONEY        NOT NULL,
    sales_cost          MONEY        NULL,
    margin              MONEY        NULL,
    discount_amount     MONEY        NOT NULL,
    discount_pct        DECIMAL(5,2) NOT NULL,
    sales_timestamp     DATETIME     NOT NULL,
    source_system_code  INT          NOT NULL,
    create_timestamp    DATETIME     NOT NULL,
    update_timestamp    DATETIME     NOT NULL,

    CONSTRAINT pk_factsales
        PRIMARY KEY (sales_key),

    CONSTRAINT uq_factsales_order_line
        UNIQUE (order_id, line_number),

    CONSTRAINT fk_factsales_date
        FOREIGN KEY (sales_date_key)
        REFERENCES gold.DimDate (date_key),

    CONSTRAINT fk_factsales_customer
        FOREIGN KEY (customer_key)
        REFERENCES gold.DimCustomer (customer_key),

    CONSTRAINT fk_factsales_employee
        FOREIGN KEY (employee_key)
        REFERENCES gold.DimEmployee (employee_key),

    CONSTRAINT fk_factsales_channel
        FOREIGN KEY (channel_key)
```

```
    REFERENCES gold.DimChannel (channel_key),

  CONSTRAINT fk_factsales_product
    FOREIGN KEY (product_key)
    REFERENCES gold.DimProduct (product_key),

  CONSTRAINT fk_factsales_store
    FOREIGN KEY (store_key)
    REFERENCES gold.DimStore (store_key)
);
```

5.2.2 FactInventorySnapshot

FactInventorySnapshot is a periodic snapshot fact table capturing one row per product, per store, per day. It serves the Inventory domain reporting requirements FR-INV-001 through to FR-INV-001, including daily stock level monitoring, shrinkage calculation, and inventory value reporting.

A surrogate primary key inventory_snapshot_key is used for consistency with the surrogate-key standard applied across all RetailSphere fact tables. Unlike the FactSales, the natural key here is genuinely unique on its own – product_key, store_key, and snapshot_date_key together identify exactly one row, since only one snapshot is taken per product per store per day. This combination is declared as a unique constraint to prevent duplicate snapshot loads, mirroring the degenerate-dimension pattern established in FactSales.

quantity_on_hand, quantity_on_order, and quantity_reserved are semi-additive measures and must be averaged, not summed, across date ranges in downstream reporting – summing stock-on-hand across a week produces a meaningless total. total_inventory_value is not carried as a physical column; it is derived I the dbt Gold model as quantity_on_hand x unit_cost, consistent with Fabric does not enforce, dbt does pattern established in this section's introduction.

```
-- ========================================
-- FactInventorySnapshot
-- Periodic snapshot grain — one row per
-- product, per store, per day
-- Sources: SAP ECC, Oracle MICROS POS
-- ========================================

CREATE TABLE gold.FactInventorySnapshot (
  inventory_snapshot_key  BIGINT       NOT NULL,
  snapshot_date_key       INT          NOT NULL,
  product_key             INT          NOT NULL,
  store_key               INT          NOT NULL,
  supplier_key            INT          NOT NULL,
  quantity_on_hand        INT          NOT NULL,
  quantity_on_order       INT          NOT NULL,
  quantity_reserved       INT          NOT NULL,
  unit_cost               MONEY        NOT NULL,
  snapshot_timestamp      DATETIME     NOT NULL,
  source_system_code      INT          NOT NULL,
  create_timestamp        DATETIME     NOT NULL,
  update_timestamp        DATETIME     NOT NULL,

  CONSTRAINT pk_factinventorysnapshot
```

```
    PRIMARY KEY (inventory_snapshot_key),

  CONSTRAINT uq_factinventorysnapshot_grain
    UNIQUE (product_key, store_key, snapshot_date_key),

  CONSTRAINT fk_factinventorysnapshot_date
    FOREIGN KEY (snapshot_date_key)
    REFERENCES gold.DimDate (date_key),

  CONSTRAINT fk_factinventorysnapshot_product
    FOREIGN KEY (product_key)
    REFERENCES gold.DimProduct (product_key),

  CONSTRAINT fk_factinventorysnapshot_store
    FOREIGN KEY (store_key)
    REFERENCES gold.DimStore (store_key),

  CONSTRAINT fk_factinventorysnapshot_supplier
    FOREIGN KEY (supplier_key)
    REFERENCES gold.DimSupplier (supplier_key)
);
```

5.2.3 FactReplenishment

FactReplenishment is an accumulating snapshot fact table capturing one row per replenishment order per store per product. It tracks the full replenishment pipeline from reorder through DC pick, dispatch, and store receipt, enabling pipeline efficiency and supplier lead time analysis.

Unlike FactSales and FactInventorySnapshot, which are insert-only, FactReplenishment rows are updated in place as the order progresses through its lifecycle. Only **reorder_date_key** – the date that establishes the row's grain – is a foreign key to DimDate. **dc_pick_date**, **dispatch_date**, and **store_receipt_date** are stored as plain date columns and remain NULL until that pipeline stage completes. **days_to_pick**, **days_to_dispactch**, **days_to_receipt**, and **total_days_pipeline** are recalculated and stored at each stage completion, and **order_status** is derived from which milestone date columns are populated.

```
-- ============================================
-- FactReplenishment
-- Accumulating snapshot grain — one row per
-- replenishment order, per store, per product
-- Sources: SAP ECC
-- ============================================

CREATE TABLE gold.FactReplenishment (
  replenishment_key     BIGINT       NOT NULL,
  reorder_date_key      INT          NOT NULL,
  product_key           INT          NOT NULL,
  store_key             INT          NOT NULL,
  supplier_key          INT          NOT NULL,
  order_id              VARCHAR(50)  NOT NULL,
  dc_pick_date          DATE         NULL,
  dispatch_date         DATE         NULL,
  store_receipt_date    DATE         NULL,
```

```
    days_to_pick            INT          NULL,
    days_to_dispatch        INT          NULL,
    days_to_receipt         INT          NULL,
    total_days_pipeline     INT          NULL,
    quantity                INT          NOT NULL,
    order_status            VARCHAR(20)  NOT NULL,
    source_system_code      INT          NOT NULL,
    create_timestamp        DATETIME     NOT NULL,
    update_timestamp        DATETIME     NOT NULL,

    CONSTRAINT pk_factreplenishment
      PRIMARY KEY (replenishment_key),

    CONSTRAINT uq_factreplenishment_grain
      UNIQUE (order_id, store_key, product_key),

    CONSTRAINT fk_factreplenishment_date
      FOREIGN KEY (reorder_date_key)
      REFERENCES gold.DimDate (date_key),

    CONSTRAINT fk_factreplenishment_product
      FOREIGN KEY (product_key)
      REFERENCES gold.DimProduct (product_key),

    CONSTRAINT fk_factreplenishment_store
      FOREIGN KEY (store_key)
      REFERENCES gold.DimStore (store_key),

    CONSTRAINT fk_factreplenishment_supplier
      FOREIGN KEY (supplier_key)
      REFERENCES gold.DimSupplier (supplier_key)
);
```

5.2.4 FactProcurement

FactProcurement is an accumulating snapshot fact table capturing one row per purchase order line item per supplier. It tracks the full procurement pipeline from PO raised through GRN receipt, invoice matching, and payment, enabling supplier on-time delivery, invoice accuracy, price variance, and days payable outstanding analysis as required by FR-PRO-001 through FR-PRO-005.

As with FactReplenishment, only po_date_key is a foreign key to DimDate – it establishes the row's grain. grn_recrived_date, invoice_received_date, invoice_matched_date, invoice_paid_date and delivery_date are stored as plain date columns and remain NULL until each stage completes. Duration columns (days_to_deliver, days_to_invoice, days_to_match, days_to_payment, total_days_pipeline) and po_status are recalculated and stored at each stage completion, consistent with the update-in-place behavior established in FactReplenishment. price_variance and price_variance_pct are not stored physical columns; they are derived in the dbt Gold model from unit_cost and contracted_rate.

A single purchase order raised against a supplier may cover multiple product lines. purchase_order_id alone is therefore insufficient to guarantee row uniqueness – the unique constraint is declared on purchase_order_id combined with product_key, mirroring the multi-line-order pattern already established in FactReplenishment.

```
-- ============================================
-- FactProcurement
-- Accumulating snapshot grain — one row per
-- purchase order line item, per supplier
-- Sources: SAP ECC
-- ============================================

CREATE TABLE gold.FactProcurement (
    procurement_key         BIGINT        NOT NULL,
    po_date_key             INT           NOT NULL,
    product_key             INT           NOT NULL,
    supplier_key            INT           NOT NULL,
    purchase_order_id       VARCHAR(50)   NOT NULL,
    grn_received_date       DATE          NULL,
    invoice_received_date   DATE          NULL,
    invoice_matched_date    DATE          NULL,
    invoice_paid_date       DATE          NULL,
    delivery_date           DATE          NOT NULL,
    days_to_delivery        INT           NULL,
    days_to_invoice         INT           NULL,
    days_to_match           INT           NULL,
    days_to_payment         INT           NULL,
    total_days_pipeline     INT           NULL,
    quantity                INT           NOT NULL,
    unit_cost               MONEY         NULL,
    contracted_rate         MONEY         NOT NULL,
    total_order_value       MONEY         NULL,
    po_status               VARCHAR(20)   NOT NULL,
    source_system_code      INT           NOT NULL,
    create_timestamp        DATETIME      NOT NULL,
    update_timestamp        DATETIME      NOT NULL,

    CONSTRAINT pk_factprocurement
        PRIMARY KEY (procurement_key),

    CONSTRAINT uq_factprocurement_grain
        UNIQUE (purchase_order_id, product_key),

    CONSTRAINT fk_factprocurement_date
        FOREIGN KEY (po_date_key)
        REFERENCES gold.DimDate (date_key),

    CONSTRAINT fk_factprocurement_product
        FOREIGN KEY (product_key)
        REFERENCES gold.DimProduct (product_key),

    CONSTRAINT fk_factprocurement_supplier
        FOREIGN KEY (supplier_key)
        REFERENCES gold.DimSupplier (supplier_key)
);
```

5.2.5 FactContactHistory

FactContactHistory is an accumulating fact table capturing one row per customer per campaign per channel per contact attempt. It tracks the communication lifecycle from sent through delivery, opened, clicked, bounced, and unsubscribed, and enforces POPIA compliance through the 4-contact monthly cap and opt-out suppression logic (ADR-007).

DM-001 does not include an attempt-sequencing column for this table. In its absence, the unique constraint is built from the full dimension combination plus grain date – **customer_key**, **campaign_key**, **channel_key**, and **contact_attempt_date_key** - on the documented assumption of at most one contact attempt per combination per day.

Suppressed contact attempts are recorded as rows in this table, not excluded from it: **was_suppressed** and **suppression_reason** capture attempts blocked before sending (opt-out, monthly cap reached), preserving the audit trail POPIA compliance requires. **Sent_date** is NULL both when an attempt is suppressed and when it simply hasn't been sent yet – these are two different NULL states distinguished by **was_suppressed,** not by the date column itself. **response_status** is stored column reflecting the furthest stage reached as of last update, consistent with the status-column pattern established in FactReplenishment and FactProcurement.

```
-- ============================================
-- FactContactHistory
-- Accumulating snapshot grain — one row per
-- customer, per campaign, per channel, per
-- contact attempt (assumes max 1/day — see
-- Open Item: DM-001 attempt-sequencing gap)
-- Sources: Marketing Cloud, Meta, GA4
-- ============================================

CREATE TABLE gold.FactContactHistory (
    contact_history_key      BIGINT       NOT NULL,
    contact_attempt_date_key  INT         NOT NULL,
    customer_key             INT         NOT NULL,
    campaign_key             INT         NOT NULL,
    channel_key              INT         NOT NULL,
    response_status          VARCHAR(20)   NOT NULL,
    suppression_reason       VARCHAR(30)   NULL,
    was_suppressed           BIT         NOT NULL,
    sent_date                DATE        NULL,
    delivered_date           DATE        NULL,
    opened_date              DATE        NULL,
    clicked_date             DATE        NULL,
    bounced_date             DATE        NULL,
    unsubscribed_date        DATE        NULL,
    source_system_code       INT         NOT NULL,
    create_timestamp         DATETIME    NOT NULL,
    update_timestamp         DATETIME    NOT NULL,

    CONSTRAINT pk_factcontacthistory
        PRIMARY KEY (contact_history_key),

    CONSTRAINT uq_factcontacthistory_grain
```

```
    UNIQUE (customer_key, campaign_key, channel_key, contact_attempt_date_key),

  CONSTRAINT fk_factcontacthistory_date
    FOREIGN KEY (contact_attempt_date_key)
    REFERENCES gold.DimDate (date_key),

  CONSTRAINT fk_factcontacthistory_customer
    FOREIGN KEY (customer_key)
    REFERENCES gold.DimCustomer (customer_key),

  CONSTRAINT fk_factcontacthistory_campaign
    FOREIGN KEY (campaign_key)
    REFERENCES gold.DimCampaign (campaign_key),

  CONSTRAINT fk_factcontacthistory_channel
    FOREIGN KEY (channel_key)
    REFERENCES gold.DimChannel (channel_key)
);
```

5.2.6 FactFinance

FactFinance is a transaction fact table capturing one row per GL journal line item posted in SAP, enabling Johan's team to produce month-end close packs and support external auditor requirements. It uses two role-playing date dimensions – **posted_date_key** for the date the entry was posted to the GL, and **transaction_date_key** for the actual business transaction date. Comparing these two dates enables automated backdated posting detection within one hour, as required by FR-FIN-005.

Note: DM-001 section 3 classifies FactFinance as an accumulating snapshot fact table; this contradicts section 9.1's own description and the table's structure, which shows no stage-progression or status columns and behaves identically to the insert-once pattern established in FactSales. PDD-001 proceeds on the section 9.1 classification (transaction fact). DM-001 section 3 to be corrected in v1.1.

journal_entry_id groups the debt and credit lines belonging to one journal entry; **line_number** identifies each line within it, starting at 1. Together they form the unique constraint, mirroring the **order_id + line_number** pattern established in FactSales. **debit_amount** and **credit_amount** are mutually exclusive per line – one always holds the transaction value, the other is 0, following standard double-entry bookkeeping convention rather than the NULL-means-unknown convention used elsewhere in this platform, since both values are always genuinely known at the moment of posting.

fiscal_period_key, listed in DM-001 section 9.1 as an FK to DimDate fiscal_period column, has been omitted from this DDL. **fiscal_period** is an attribute of DimDate, not a key – it's already accessible via the existing **posted_date_key** join, making a separate FK both structurally invalid and redundant. Flagged for removal in DM-001 v1.1.

```
-- ============================================
-- FactFinance
-- Transaction grain — one row per GL journal
-- line item
-- Sources: SAP ECC General Ledger
-- ============================================
```

```
CREATE TABLE gold.FactFinance (
    finance_key          BIGINT       NOT NULL,
    posted_date_key      INT          NOT NULL,
    transaction_date_key INT          NOT NULL,
    account_key          INT          NOT NULL,
    store_key            INT          NOT NULL,
    journal_entry_id     VARCHAR(50)  NOT NULL,
    line_number          INT          NOT NULL,
    debit_amount         MONEY        NOT NULL,
    credit_amount        MONEY        NOT NULL,
    is_backdated         BIT          NOT NULL,
    source_system_code   INT          NOT NULL,
    create_timestamp     DATETIME     NOT NULL,
    update_timestamp     DATETIME     NOT NULL,

    CONSTRAINT pk_factfinance
        PRIMARY KEY (finance_key),

    CONSTRAINT uq_factfinance_grain
        UNIQUE (journal_entry_id, line_number),

    CONSTRAINT fk_factfinance_posted_date
        FOREIGN KEY (posted_date_key)
        REFERENCES gold.DimDate (date_key),

    CONSTRAINT fk_factfinance_transaction_date
        FOREIGN KEY (transaction_date_key)
        REFERENCES gold.DimDate (date_key),

    CONSTRAINT fk_factfinance_account
        FOREIGN KEY (account_key)
        REFERENCES gold.DimAccount (account_key),

    CONSTRAINT fk_factfinance_store
        FOREIGN KEY (store_key)
        REFERENCES gold.DimStore (store_key)
);
```

5.2.7 FactSupplierContract

FactSupplierContact resolves the many-to-many relationship between suppliers and products. One row per supplier per product per contract captures contracted rates, lead times, and payment terms extracted from PDF contracts via ADR-003. This table is the source of **contracted_rate** used in FR-PRO-003's price variance calculation in FactProcurement.

Note: DM-001 section 3 classifies FactSupplierContract as an accumulating snapshot fact table. On review, it fits none of the three types defined in section 3 cleanly – it lacks the defined pipeline stages of a true accumulating snapshot, but permits in-place updates when contract terms are amended, unlike a standard transaction fact. PDD-001 treats it as the closest available fit (transaction fact, atypical – amendments permitted). Flagged for DM-001 v1.1: either document a fourth fact type or add an explicit exception note.

A single supplier contract PDF may cover multiple products under one negotiated agreement. **contract_id** alone is therefore insufficient to guarantee row uniqueness – the unique constraint is **contract_id + product_key**, consistent with the multi-line-agreement patter established in FactProcurement and FactReplenishment.

contract_end_date is NULL for open-ended contracts. This is a distinct NULL convention from the "not yet known" milestone dates used in the accumulating snapshot tables – here NULL means permanently not applicable, not pending.

```
-- ============================================
-- FactSupplierContract
-- One row per supplier, per product, per
-- contract — resolves supplier/product M:M
-- Sources: ADR-003 PDF extraction pipeline
-- ============================================

CREATE TABLE gold.FactSupplierContract (
    supplier_contract_key    BIGINT       NOT NULL,
    contract_start_date_key   INT         NOT NULL,
    supplier_key              INT         NOT NULL,
    product_key               INT         NOT NULL,
    contract_id               VARCHAR(50)   NOT NULL,
    contract_end_date           DATE        NULL,
    quantity                  INT         NOT NULL,
    contracted_rate             MONEY       NOT NULL,
    lead_time                   INT         NOT NULL,
    payment_terms                 VARCHAR(20)   NOT NULL,
    source_system_code              INT         NOT NULL,
    create_timestamp                DATETIME    NOT NULL,
    update_timestamp                  DATETIME    NOT NULL,

    CONSTRAINT pk_factsuppliercontract
        PRIMARY KEY (supplier_contract_key),

    CONSTRAINT uq_factsuppliercontract_grain
        UNIQUE (contract_id, product_key),

    CONSTRAINT fk_factsuppliercontract_date
        FOREIGN KEY (contract_start_date_key)
        REFERENCES gold.DimDate (date_key),

    CONSTRAINT fk_factsuppliercontract_supplier
        FOREIGN KEY (supplier_key)
        REFERENCES gold.DimSupplier (supplier_key),

    CONSTRAINT fk_factsuppliercontract_product
        FOREIGN KEY (product_key)
        REFERENCES gold.DimProduct (product_key)
);
```

6. Indexes and Constraints

Traditional index tuning — clustered index selection, covering indexes, filtered indexes — does not apply to RetailSphere's Gold layer. Microsoft Fabric Warehouse stores all tables as Delta Lake, and physical storage optimization is managed by the platform rather than declared through T-SQL DDL. There is no **CREATE INDEX** statement in Fabric Warehouse.

The underlying optimization mechanism is **V-Order**, a write-time Parquet optimization (sorting, row group distribution, encoding, and compression) that improves read performance across Fabric's query engines, particularly for read-heavy, repeated-scan workloads such as Power BI dashboarding against the Gold layer — the dominant access pattern for this platform. Fabric Warehouse developers interact with tables exclusively through T-SQL; there is no exposed Spark/OPTIMIZE surface for manual tuning, as this is a Lakehouse/Spark-only capability.

PRIMARY KEY, **UNIQUE**, and **FOREIGN KEY** constraints are declared throughout the Section 5 DDL for documentation, lineage, and BI-tool metadata purposes only — Fabric Warehouse does not enforce them. Actual data integrity enforcement is handled by dbt tests in the Silver layer transformation pipeline, as established in Section 5.2's introduction.

7. Partitioning Strategy

Traditional partitioning — a partition function and scheme dividing a table into physically separate segments for query pruning and archival — is not applicable to RetailSphere's Gold layer, because **Microsoft Fabric Warehouse does not currently support table partitioning.** This is a platform limitation, not a design choice: partitioning is on Microsoft's product backlog with no committed delivery date at the time of writing (August 2026).

In place of partitioning, Fabric Warehouse offers **data clustering**, declared via a **CLUSTER BY** clause on **CREATE TABLE** or **CREATE TABLE AS SELECT**. Clustering physically co-locates rows with similar values in the clustering column(s) during ingestion, improving read performance for queries that filter on those columns — similar intent to partitioning, but without folder-level pruning or partition-level purge/archival capability.

For RetailSphere's two highest-growth fact tables, clustering is recommended on the grain date column:

- **FactSales** — cluster on **sales_date_key**. The highest-volume table in the platform; most Sales domain queries (FR-SAL-001 through FR-SAL-005) filter by date range.
- **FactInventorySnapshot** — cluster on snapshot_date_key. Grows by a fixed volume daily; stock-position and trend queries are consistently date-filtered.

Per Fabric guidance, clustering columns should favour lower cardinality to avoid excessive file fragmentation — date keys at daily grain are a reasonable fit and align with the guidance against clustering on high-cardinality columns such as raw timestamps.

8. Next Steps

The completion of PDD-001 marks the end of Step 9 — Physical Database Design. Reviewed and approved by Nomvula Mahlangu, Chief Data Officer, 23 August 2026. The following items carry forward:

Immediate:

- Publish PDD-001 and update Notion step tracker and README Step 9 status..
- (DM-001 v1.1 already published, applying five items surfaced during PDD-001 development — see DM-001's own Change Log.)

Step 10 — Data Integration Design (DID-001): Design the ADF pipeline architecture for Bronze, Silver, and Gold layer ingestion and transformation, including error handling, retry logic, and monitoring. Two specific dependencies on DID-001, raised during CDO review

- **Backdated posting notification (FR-FIN-005):** PDD-001 captures the fact needed to detect backdated postings (**is_backdated** on FactFinance). The notification mechanism — who is alerted and within what SLA — does not yet exist and is a DID-001 dependency, not a PDD-001 deliverable.
- **Data quality enforcement:** dbt enforcement of the constraints declared throughout Section 5 (PK, unique, FK) becomes concrete only once DID-001 and Silver layer dbt models are built. Until then, Fabric's informational-only constraints provide no actual protection against bad data reaching Gold — this is acceptable only because RetailSphere has not yet entered build phase (Steps 13+); it must not be assumed to be in place once build begins without explicit dbt test coverage.

Outstanding items requiring resolution before build (Steps 13+):

- FactContactHistory same-day attempt-sequencing: validate against actual Marketing Cloud send/retry behaviour before the unique constraint can be considered final (Section 5.2.5). Flagged by CDO review as an audit-defensibility risk if unresolved — if same-day retries prove possible, this requires a DM-001 structural revision (attempt-sequencing column), not a PDD-001 fix.
- ADR-002, ADR-003, ADR-006, ADR-007 sign-offs remain in progress per DM-001's own outstanding items list — these affect Source-to-Target Mapping work still deferred to build phase per STM-001 section 5.

9. Document Change Log

Version	Date	Author	Description
1.0 - Draft	23 August 2026	Thozamile Mbalo	Initial PDD-001 release. Physical DDL for 9 dimension tables and 7 fact tables across 6 business domains, translated from DM-001 v1.0. Indexes and Constraints (Section 6) and Partitioning Strategy (Section 7) addressed as platform-specific: Fabric Warehouse constraints declared as informational metadata only (dbt-enforced); native indexing not applicable (Delta Lake/V-Order managed);

			native partitioning not currently supported by platform, data clustering substituted on FactSales and FactInventorySnapshot. DM-001 v1.1 corrections (FactFinance classification, fiscal_period_key removal, FactSupplierContract type note, stale Synapse partitioning reference) identified during this work — to be published as a follow-up revision.
1.0 – Approved	23 August 2026	Reviewed by Nomvula Mahlangu, Chief Data Officer	Approved following review. Two findings incorporated into Section 8 (Next Steps): backdated posting notification mechanism (FR-FIN-005) confirmed as a DID-001 dependency, not yet built; dbt-based data quality enforcement confirmed as dependent on Silver layer models not yet built. FactContactHistory attempt-sequencing gap (Section 5.2.5) reaffirmed as an open item requiring validation before build.

— End of Document —